

PROTOCOLES IOT – MQTT / HTTP / COAP (ETAPES 2 MQTT)

Cette fiche est conçue spécialement dans le cadre de mes examens du BTS CIEL IR Option A – Épreuve E6, à des fins de révision. Elle peut également servir à toute personne souhaitant comprendre les protocoles utilisés dans les systèmes IoT (Internet Of Things).

Dans l'épreuve E6, le jury peut poser des questions comme :

- Pourquoi utiliser MQTT pour l'IoT ?
- Pourquoi HTTP n'est-il pas adapté aux capteurs ?
- Quelle est la différence entre MQTT et CoAP ?
- Comment un capteur envoie ses données vers un serveur ?
- Quel protocole consomme le moins d'énergie ?

Beaucoup d'étudiants connaissent les protocoles web mais ne comprennent pas les contraintes des objets connectés.

L'étude des protocoles IoT se fera en 4 étapes progressives :

1. Les limites du modèle web classique
2. Le protocole HTTP
3. Le protocole MQTT
4. Le protocole CoAP

Ici on continue avec **le protocole MQTT**.

Le protocole MQTT est aujourd'hui l'un des protocoles les plus utilisés dans l'IoT, il répond aux contraintes du protocole HTTP/IoT. Il a été créé pour permettre aux objets ayant peu d'énergie, peu de puissance de calcul et un réseau instable de communiquer efficacement.

Contrairement au protocole HTTP qui utilise un modèle client/serveur, MQTT lui utilise un modèle publish/subscribe.

Cela signifie que les appareils n'envoient pas directement leurs données aux autres appareils. Ils envoient leurs messages à un serveur central appelé **broker**.

Schéma simplifié : Publisher → (publish sur topic) → Broker → (envoi aux abonnés) → Subscriber

Le **Publisher** est l'appareil qui **envoie une donnée** par exemple le capteur de température.

Le **Subscriber** est l'appareil qui **reçoit la donnée** par exemple une application mobile ou une dashboard IoT.

Le **Broker** est le serveur central qui **distribue les messages**.

Le **topic** est un canal de communication comme une étiquette qui catégorise les messages. Il permet au broker de savoir à qui envoyer le message. Un **topic** peut ressembler à ça : capteurs/temp | capteurs/humidite. Un **subscriber** s'abonne à un topic, et reçoit tous les messages publiés sur **ce topic**.

Exemple concret avec Mosquitto

- Publisher (envoi d'un message) : `mosquitto_pub -h localhost -t test/topic -m "Bonjour MQTT"`
- Subscriber (écoute du topic) : `mosquitto_sub -h localhost -t test/topic`

Résultat : Le subscriber reçoit le message "Bonjour MQTT" dès qu'il est publié

Le fichier *mosquitto.conf* permet de configurer le **broker MQTT**. Il définit les paramètres de fonctionnement comme le port d'écoute, les autorisations d'accès (authentification ou accès anonyme), ainsi que d'autres options de configuration.

Par exemple :

- `mosquitto_sub -h localhost -p 1883 -t test/topic`
- `mosquitto_pub -h localhost -p 1883 -t test/topic -m "Hello"`

Le protocole est conçu pour envoyer le minimum de données possible.

Cela réduit :

- La consommation réseau
- La consommation d'énergie

Les capteurs n'ont pas besoin de savoir qui reçoit les données. Ils envoient simplement la donnée au broker. Il peut envoyer une donnée qui sera reçue par 1 client, 10 clients, 1000 clients sans modifier le capteur.

Question sur le protocole MQTT :

Pourquoi MQTT est plus adapté que HTTP pour l'IoT ?

MQTT pour l'IoT est plus adapté que HTTP parce que les messages sont plus légers et le modèle publish/subscribe est plus flexible, la consommation d'énergie est plus faible.

Explique le rôle du broker MQTT.

Le broker est le serveur central qui reçoit les messages des capteurs et les distribue aux clients abonnés.

Pourquoi MQTT est scalable ?

Le capteur n'a pas besoin de connaître les clients, le broker distribue à 10, 100 ou 1000 clients et un seul message peut être envoyé à beaucoup de récepteurs.

Si tu as 1000 capteurs IoT dans une usine, pourquoi utiliser MQTT avec un broker plutôt que 1000 connexions HTTP directes vers le serveur ?

MQTT est plus adapté pour l'envoi de petites données comme celle de capteurs, c'est un protocole plus léger publish/subscribe ça veut dire que les données sont envoyées sur un broker qui lui va ensuite envoyer les données aux appareils grâce à ça les capteurs n'ont pas besoin de connaître tous les destinataires tandis que HTTP pour ce type de système n'est pas adapté car trop lourd il utilise trop de requêtes contenant headers cookies métadonnées pour des données légères c'est surdimensionné.

Avec HTTP : 1000 capteurs → 1000 connexions serveur

Avec MQTT : 1000 capteurs → 1 broker

Le broker redistribue les données. Donc le système est beaucoup plus scalable.

Quelle est le port pour MQTT sécurisé et MQTT avec TLS/SSL.

1883 → pour MQTT non sécurisé (sans chiffrement)

8883 → pour MQTT sécurisé

Réalisez un diagramme de déploiement UML permettant de représenter fidèlement cette architecture. Le diagramme doit faire apparaître les éléments suivants.

Contexte

Une entreprise industrielle souhaite superviser en temps réel ses lignes de production. Chaque ligne est équipée d'un système embarqué chargé de collecter des données physiques (température, vibrations, nombre de cycles, etc.) et de les transmettre à une infrastructure centralisée pour exploitation.

Architecture technique cible

- *Nœud embarqué : une carte embarquée exécutant Zephyr OS, connectée au réseau local via Ethernet, assure l'acquisition des mesures physiques.*
- *Transmission des données : les mesures sont formatées localement, puis publiées sur un broker MQTT (Mosquitto).*
- *Infrastructure réseau : le broker MQTT centralise les données issues des différentes cartes.*
- *Supervision : un agent Telegraf (abonné aux topics MQTT) collecte les données et les stocke dans une base InfluxDB.*
- *Visualisation : les opérateurs de production accèdent aux données via des tableaux de bord Grafana.*
- *Administration : un administrateur réseau supervise l'état de la plateforme de supervision.*

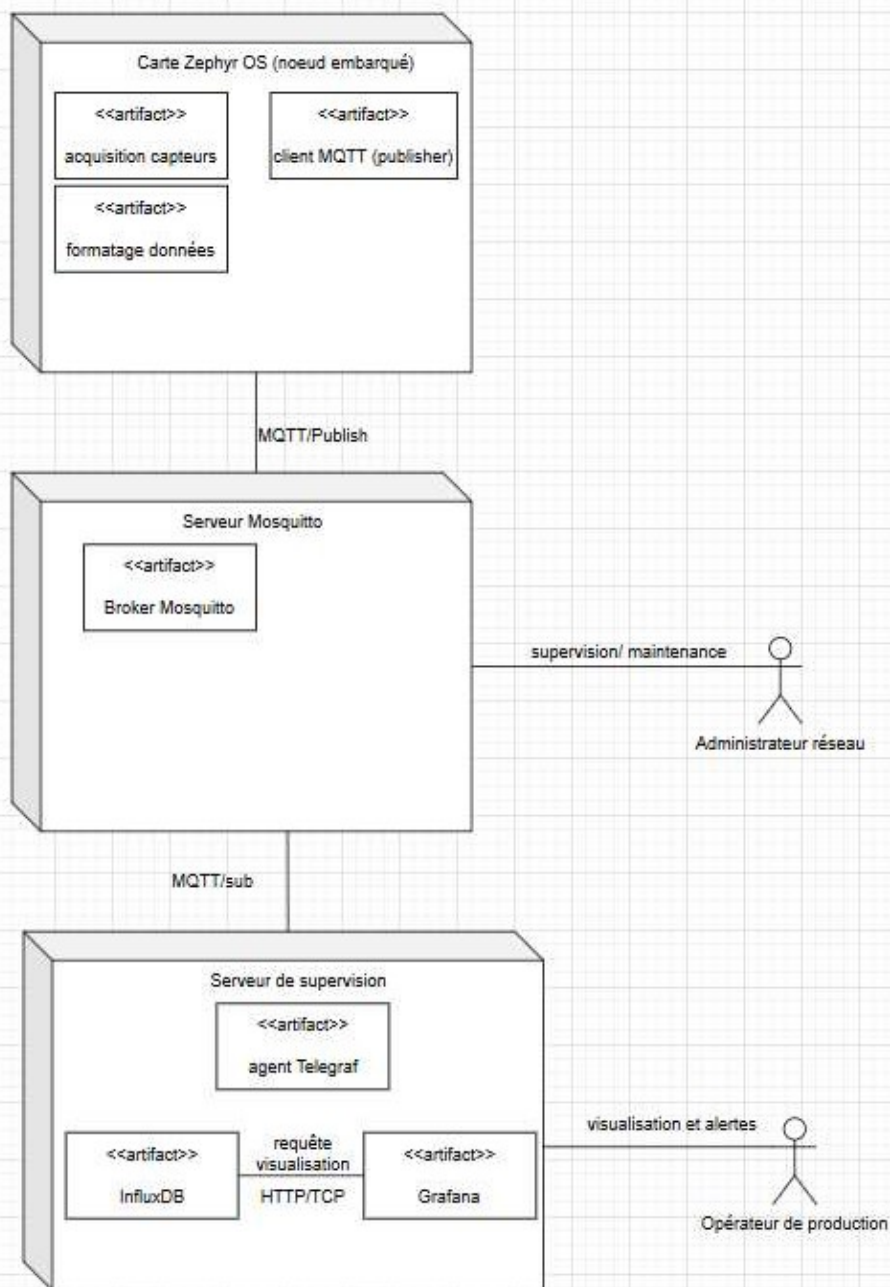
Les nœuds matériels : carte Zephyr OS (nœud embarqué), serveur Mosquitto, serveur de supervision.

Les artefacts logiciels déployés sur chaque nœud : sur la carte Zephyr → acquisition capteurs, formatage données, client MQTT (publisher), sur le serveur Mosquitto → broker Mosquitto, sur le serveur de supervision → agent Telegraf, InfluxDB, Grafana.

Les flux réseau utilisés : MQTT (entre carte embarquée et broker), MQTT (entre broker et Telegraf), requête HTTP/TCP (entre Grafana et InfluxDB).

Les acteurs humains : opérateur de production (visualisation et alertes), administrateur réseau (supervision et maintenance Docker).

Les liens entre composants, avec étiquetage des échanges si pertinent (mqtt/publish, mqtt/sub, requête visualisation, etc.).



1. Broker Mosquitto

Centralise les messages publiés par les cartes embarquées. **Ne répond pas directement à Grafana** qui lit les données pour visualisation sur un Dashboard par exemple.

2. Agent Telegraf

Sur le serveur de supervision, c'est **Telegraf** qui s'abonne aux topics MQTT du broker. Donc la **communication avec le broker se fait via MQTT** → mqtt/sub

3. InfluxDB toujours sur le serveur de supervision **Telegraf** envoie les données collectées à **InfluxDB** puis **Grafana** interroge InfluxDB pour visualiser les données qui utilise **HTTP/TCP** → requête visualisation